

Frontend della dashboard meteo

Vite + React + API backend + Vercel

Scopo: Portare il frontend da progetto Vite vuoto a dashboard pubblica e responsive. La guida descrive struttura, stato, componenti, contratti API, test e deploy, senza fornire il codice completo dell'applicazione.

1. Risultato finale atteso

- Campo per cercare una città o un luogo.
- Lista di risultati omonimi con regione e paese.
- Selezione del luogo corretto.
- Cinque card meteo, da oggi ai quattro giorni successivi.
- Stati di caricamento, vuoto ed errore.
- Layout accessibile e responsive.
- Frontend pubblico su Vercel collegato al backend Render.

Utente -> SearchForm -> API locations -> LocationList

Utente seleziona -> API forecast -> ForecastGrid -> 5 ForecastCard

2. Documentazione ufficiale da usare

Vite - Getting Started - <https://vite.dev/guide/>

Per creare il progetto React e capire dev server e script.

Vite - Environment Variables - <https://vite.dev/guide/env-and-mode>

Per usare VITE_API_URL e capire che le variabili VITE_ sono pubbliche.

Vite - Production Build - <https://vite.dev/guide/build>

Per verificare npm run build e la cartella dist.

React - State - <https://react.dev/learn/state-a-components-memory>

Per gestire query, risultati, loading, errore e forecast.

React - Managing State - <https://react.dev/learn/managing-state>

Per decidere dove mantenere lo stato e ridurre duplicazioni.

React - Reacting to Input - <https://react.dev/learn/reacting-to-input-with-state>

Per progettare gli stati della UI e i form.

React - Effects - <https://react.dev/learn/synchronizing-with-effects>

Da usare solo quando serve sincronizzarsi con sistemi esterni.

MDN Fetch API - https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Per richieste HTTP, response.ok e parsing JSON.

MDN Forms Accessibility - https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/How_to_structure_a_web_form

Per label, input, button e messaggi.

Vercel - Vite - <https://vercel.com/docs/frameworks/frontend/vite>

Per importare il repository, configurare build e variabili.

3. Prerequisiti

Strumento	Uso	Verifica	Atteso
Node.js LTS	Vite e npm	node --version	Versione LTS
npm	Dipendenze e script	npm --version	Comando disponibile
Git	Repository e deploy	git --version	Comando disponibile
Browser moderno	DevTools e test	Aprire DevTools	Network e Console disponibili
Backend locale	Dati reali	/api/health	HTTP 200

Dipendenza dal backend: Prima di collegare le schermate, il backend deve avere almeno /api/health e il contratto delle due API meteo definito.

4. Creazione del progetto Vite

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

1. Aprire l'indirizzo mostrato da Vite, normalmente <http://localhost:5173>.
2. Verificare che la pagina iniziale sia visibile.
3. Rimuovere progressivamente i contenuti demo.
4. Non installare librerie aggiuntive finché non emerge un bisogno concreto.

5. Struttura completa consigliata

```
frontend/
|-- public/
|-- src/
|   |-- api/
|   |   `-- weatherApi.js
|   |-- components/
|   |   |-- SearchForm.jsx
|   |   |-- LocationList.jsx
|   |   |-- ForecastCard.jsx
```

```
| | |-- ForecastGrid.jsx
| | |-- LoadingMessage.jsx
| | `-- ErrorMessage.jsx
| |-- pages/
| | `-- WeatherDashboard.jsx
| |-- utils/
| | |-- weatherIcons.js
| | `-- dateFormat.js
| |-- App.jsx
| |-- main.jsx
| `-- index.css
|-- .env
|-- .env.example
|-- .gitignore
|-- package.json
`-- vite.config.js
```

weatherApi.js - Unico punto che conosce l'URL e il formato HTTP del backend.

WeatherDashboard.jsx - Coordina stato, richieste e passaggio dei dati ai componenti.

SearchForm.jsx - Gestisce testo, submit e disabilitazione durante il caricamento.

LocationList.jsx - Presenta risultati omonimi e comunica la selezione.

ForecastGrid.jsx - Organizza le cinque card.

ForecastCard.jsx - Mostra un singolo giorno.

weatherIcons.js - Associa weatherCode a icona o simbolo.

dateFormat.js - Formatta date in italiano senza duplicare logica.

6. Contratto condiviso con il backend

Prima di scrivere i componenti, annotare il formato esatto delle risposte. Il frontend non deve indovinare i campi.

Ricerca luoghi

```
GET {VITE_API_URL}/weather/locations?q=Roma
```

- locations: array
- id
- name
- state opzionale
- country
- countryCode
- latitude
- longitude
- timezone

Previsione

```
GET {VITE_API_URL}/weather/forecast?lat=...&lon=...
```

- location: coordinate e timezone

- forecast: array di 5 elementi
- date
- weatherCode
- description
- temperatureMin
- temperatureMax
- precipitationProbability
- windSpeed

Regola: Qualsiasi modifica ai nomi dei campi deve essere concordata e applicata sia nel backend sia nel frontend.

7. Configurazione delle variabili Vite

Creare `.env` e `.env.example` nella radice frontend.

```
VITE_API_URL=http://localhost:8080/api
```

- Vite espone al browser soltanto variabili con prefisso `VITE_`.
- Queste variabili sono pubbliche e non devono contenere segreti.
- Dopo una modifica a `.env`, riavviare `npm run dev`.
- `weatherApi.js` deve leggere `import.meta.env.VITE_API_URL`.
- I componenti non devono contenere URL hard-coded.

8. Progettare gli stati della dashboard

Stato	Tipo	Significato
<code>query</code>	<code>string</code>	Testo inserito
<code>locations</code>	<code>array</code>	Risultati della ricerca
<code>selectedLocation</code>	<code>object/null</code>	Luogo scelto
<code>forecast</code>	<code>array</code>	Cinque giorni
<code>loadingLocations</code>	<code>boolean</code>	Ricerca in corso
<code>loadingForecast</code>	<code>boolean</code>	Forecast in corso
<code>error</code>	<code>string/null</code>	Errore leggibile
<code>view</code>	<code>derivato</code>	Stato visuale attuale

Evitare stati duplicati: per esempio non salvare separatamente `hasResults` se può essere derivato da `locations.length`. La documentazione React raccomanda di mantenere lo stato minimo necessario.

9. Creare il client API

weatherApi.js deve nascondere fetch ai componenti. Il file deve contenere una funzione comune di richiesta e due funzioni di dominio.

- Costruire l'URL a partire da VITE_API_URL.
- Eseguire fetch.
- Tentare il parsing JSON.
- Controllare response.ok, perché fetch non genera automaticamente un errore per 400 o 500.
- Usare il messaggio del backend quando disponibile.
- Generare un messaggio chiaro quando il backend non è raggiungibile.
- Creare searchLocations(query).
- Creare loadForecast(latitude, longitude).
- Codificare i parametri tramite URLSearchParams.

Test isolato: Prima dei componenti, richiamare temporaneamente le funzioni dalla console o da un piccolo handler e controllare DevTools > Network.

10. Implementare il form di ricerca

- Campo input di tipo search.
- Label sempre associata tramite htmlFor/id.
- Valore controllato da React.
- Submit tramite form, non soltanto click.
- Trim del testo.
- Rifiuto sotto due caratteri.
- Pulsante disabilitato durante la ricerca.
- Invio tramite Enter supportato.
- Messaggio di validazione vicino al campo.

Il form non deve conoscere come vengono visualizzate le località; deve emettere soltanto il valore valido alla dashboard.

11. Gestire la ricerca delle località

5. Azzerare errore precedente.
6. Azzerare forecast precedente se parte una nuova ricerca.
7. Attivare loadingLocations.
8. Chiamare searchLocations.
9. Salvare locations.
10. Se array vuoto, mostrare "Nessun luogo trovato".
11. Disattivare loading nel finally.
12. Non selezionare automaticamente il primo risultato se esistono omonimi.

LocationList

- Mostrare nome, state se presente, country e countryCode.
- Usare un button per ogni scelta, non un div cliccabile.

- Creare una key stabile con id oppure coordinate.
- Evidenziare focus da tastiera.
- Inviare l'intero oggetto location al callback di selezione.

12. Caricare e mostrare il forecast

13. Salvare selectedLocation.
14. Nascondere o svuotare la lista risultati.
15. Attivare loadingForecast.
16. Chiamare loadForecast con coordinate.
17. Verificare che forecast sia un array.
18. Verificare che contenga cinque elementi; se no, mostrare errore controllato.
19. Salvare il forecast.
20. Disattivare loading nel finally.

ForecastCard

- Giorno della settimana in italiano.
- Data leggibile.
- Etichetta Oggi sulla prima card.
- Descrizione meteo.
- Temperatura minima e massima con unità °C.
- Probabilità di precipitazione con %.
- Vento con unità coerente al backend.
- Icona o simbolo associato al weatherCode.
- Testo alternativo equivalente alla descrizione.

Mappa delle icone

- 0: sereno
- 1-2: parzialmente nuvoloso
- 3: coperto
- 45-48: nebbia
- 51-67 e 80-82: pioggia
- 71-77 e 85-86: neve
- 95-99: temporale
- fallback: icona generica

13. Stati visuali completi

Situazione	Cosa mostrare
Iniziale	Titolo, descrizione breve e form
Ricerca in corso	Messaggio o spinner accessibile
Risultati disponibili	Lista selezionabile
Nessun risultato	Messaggio e form ancora disponibile

Situazione	Cosa mostrare
Forecast in corso	Luogo selezionato + loading
Forecast disponibile	Titolo del luogo + 5 card
Errore	Messaggio role=alert + possibilità di riprovare

React consiglia di descrivere chiaramente gli stati visuali possibili invece di manipolare direttamente il DOM.

14. Design responsive e CSS

- Definire una larghezza massima del contenuto.
- Usare CSS Grid per forecast-grid.
- Mobile: una colonna.
- Tablet: due o tre colonne.
- Desktop: fino a cinque colonne se leggibili.
- Usare spaziatura coerente e dimensioni minime dei pulsanti.
- Non basarsi soltanto sul colore per comunicare stato.
- Garantire contrasto sufficiente.
- Mantenere visibile il focus.
- Testare zoom browser al 200%.

Non è necessario installare Tailwind o una libreria UI. CSS standard è sufficiente e riduce la complessità iniziale.

15. Accessibilità

- Un solo h1 per pagina.
- Gerarchia h2/h3 coerente.
- Label esplicita per il campo.
- Button per azioni.
- role=alert per errori importanti.
- aria-live per caricamenti o risultati dinamici, se utile.
- Alt text per icone informative; aria-hidden per icone decorative.
- Navigazione completa tramite tastiera.
- Ordine del focus naturale.
- Date e unità comprensibili nel testo.

16. Test manuali

Test	Azione	Atteso
Avvio	npm run dev	Pagina senza errori console
Validazione	Inviare "R"	Nessuna chiamata API, errore locale
Ricerca	Cercare Roma	Lista di luoghi

Test	Azione	Atteso
Omonimi	Cercare Springfield	Più risultati distinguibili
Forecast	Selezionare Roma	5 card
Nuova ricerca	Cercare Londra	Dati precedenti sostituiti
Backend spento	Fermare backend	Messaggio comprensibile
Responsive	Ridurre viewport	Nessun overflow
Tastiera	Tab e Enter	Tutte le azioni disponibili

17. Test automatici consigliati

Dopo la versione funzionante, installare Vitest e React Testing Library. Non è obbligatorio per il primo prototipo, ma migliora affidabilità e portfolio.

- SearchForm invia una query valida.
- SearchForm blocca query troppo corta.
- LocationList mostra regione opzionale correttamente.
- ForecastCard formatta data e unità.
- Dashboard mostra loading.
- Dashboard mostra errore della API.
- Dashboard mostra cinque card dopo una risposta simulata.

18. Collegamento locale al backend

21. Avviare backend sulla porta 8080.
22. Verificare /api/health.
23. Avviare frontend sulla porta 5173.
24. Controllare VITE_API_URL.
25. Cercare Roma.
26. Aprire Network e controllare richiesta locations.
27. Selezionare un luogo e controllare forecast.
28. Verificare che non vi siano errori CORS o mixed content.

Diagnosi CORS: La richiesta deve partire da `http://localhost:5173` e il backend deve autorizzare esattamente questa origine. `localhost` e `127.0.0.1` sono origini differenti.

19. Build di produzione

```
npm run build
npm run preview
```

- `npm run build` deve creare dist senza errori.
- `npm run preview` serve soltanto per controllare localmente la build.
- Aprire la preview e ripetere il flusso completo.

- Controllare che VITE_API_URL usata nella build sia corretta.
- Ricordare che le variabili Vite sono sostituite al momento della build.

20. GitHub e flusso di lavoro

```
git add .
git status
git commit -m "Create weather frontend foundation"
```

- feature/search-form
- feature/location-results
- feature/forecast-cards
- feature/loading-errors
- feature/responsive-ui
- feature/vercel-deploy

Non caricare .env. Caricare .env.example con VITE_API_URL vuoto o di esempio.

21. Deploy su Vercel

29. Caricare il repository su GitHub.
30. Aprire Vercel e scegliere Add New Project.
31. Importare il repository.
32. Se monorepo, impostare Root Directory su frontend.
33. Verificare Framework Preset: Vite.
34. Build Command: npm run build.
35. Output Directory: dist.
36. Aggiungere VITE_API_URL con URL pubblico Render seguito da /api.
37. Eseguire deploy.
38. Aprire il dominio Vercel.
39. Aggiornare FRONTEND_URL su Render con il dominio Vercel.
40. Ridistribuire o riavviare il backend.
41. Ripetere ricerca e forecast in produzione.

Ordine corretto: Puoi pubblicare prima il backend, poi il frontend, quindi tornare su Render per impostare il dominio Vercel definitivo in CORS.

22. Problemi frequenti

import.meta.env undefined - Variabile non prefissata VITE_ o server non riavviato.

Failed to fetch - Backend spento, URL errato, CORS o rete.

404 dalla API - Base URL o path duplicato /api.

Dati undefined - Contratto backend/frontend non corrispondente.

Card non aggiornate - Stato non sostituito o key errata.

Date spostate di un giorno - Parsing con fuso orario non controllato; formattare date ISO giornaliere con UTC coerente.

Vercel mostra vecchio URL - Variabile modificata senza nuovo deploy.

Mixed content - Frontend HTTPS chiama backend HTTP; usare URL HTTPS Render.

23. Sequenza di sviluppo consigliata

42. Creare Vite e ripulire demo.
43. Creare struttura cartelle.
44. Definire contratto API nel README.
45. Configurare VITE_API_URL.
46. Creare client API.
47. Creare SearchForm con validazione.
48. Collegare ricerca locations.
49. Creare LocationList.
50. Collegare selezione e forecast.
51. Creare ForecastCard e ForecastGrid.
52. Aggiungere descrizioni e icone.
53. Gestire loading, errori e lista vuota.
54. Aggiungere responsive e accessibilità.
55. Testare con backend locale.
56. Eseguire build e preview.
57. Caricare su GitHub.
58. Pubblicare su Vercel.
59. Eseguire test end-to-end online.

24. Definition of Done frontend

- ☐ npm install e npm run dev funzionano.
- ☐ Nessun errore in console all'avvio.
- ☐ Ricerca minima validata.
- ☐ Località omonime distinguibili.
- ☐ Selezione località carica forecast.
- ☐ Esattamente cinque card visualizzate.
- ☐ Loading ed errori gestiti.
- ☐ Nuova ricerca possibile.
- ☐ Layout utilizzabile su mobile e desktop.
- ☐ Navigazione da tastiera funzionante.

- ☐ npm run build completato.
- ☐ Frontend pubblico su Vercel.
- ☐ API Render chiamata tramite HTTPS.
- ☐ README contiene setup, variabili e percorso di test.